Simulation de fluides en 2D
Projet de programmation 2

Axel Pereyrol, Saurane Delrieu, Gaelle Milezi, Satine Barraux
L3 informatique
Faculté des Sciences
Université de Montpellier.

2025-2026

Résumé

Description très succinte du problème et des différentes étapes de réalisation

Table des matières

1 Présentation du Sujet **3**

2 Développements Logiciel : Conception, Modélisation, Implémentation **4**

3 Algorithmes et Structures de Données **8**

4 Analyse des résultats **11**

5 Gestion du Projet **13**

6 Bilan et Conclusions **15**

1 Présentation du Sujet

1-2 pages

- Présenter le problème étudié et le contexte dans lequel le projet se positionne.
  - projet en licence informatique L3
  - méthode de Jos Stam "Stable FLuids"
  - simulation d'un fluide en temps réel
  - intéraction utilisateur.

    Ce projet de programmation 2 a été réalisé dans le cadre de la licence informatique L3 de la faculté des sciences de Montpellier.

    L'objectif de ce project est de simmuler un fluide en 2D interactif en temps reel. Pour cela, nous avons choist d'implementer la methode de Jos Stam, "Stable Fluids", qui permet de simuler un fluide de manière stable et rapide.

    Nous avons également implémenté une interface graphique pour visualiser en temps réél l'évolution de notre projet et le fait de pouvoir intéragir avec celui-ci, notament grace a creation d'obstacle clicables et la possiblilite de modifier en temps reel different paramètres du fluide.
- Motiver l'intérêt du problème étudié par rapport à votre parcours d'études et au monde de l'informatique.

  Nous avons choisi ce projet car il nous a parru être le plus adapté à nos compétences, étant donné notre parcours en double licence mathématiques et informatique. En effet, nous avons rapidement compris les concepts de base de la simulation, ce qui nous a permis d'implémenter la méthode de Jos Stam "Stable Fluids".

  Nous avons également aimé l'idée d'implémenter une interface graphique pour visualiser en temps réél l'évolution de notre projet et le fait de pouvoir intéragir avec celui-ci. De plus, c'est avec ce projet que nous avons découvert le langage c++ et la bibliothèque OpenGl, qui sont des outils etentiels pour la simulaiton de fluides mais aussi dans de nombreux domaines tels que les jeux vidéos, la météorologie ou l'environnment.
- Présenter les différentes approches possibles pour la résolution du problème, et en particulier celle choisie.

  Pour implémenter la simulation de fluides en 2D, 2 méthodes sont possibles : la méthode eulérienne ou la méthode lagrangienne. La méthode eulérienne consiste à représenter le fluide dans une grille fixe, dans laquelle on stocke les différentes propriétés du fluide, telles que la densité, la chaleur, la vitesse, etc. La méthode lagrangienne consiste à représenter le fluide à l'aide de particules, c'est à dire que chaque particule contient les propriétés du fluide donc celui-ci est représenté comme à l'aide de particules.

  Pour l'implémentation de ce projet, nous avons choisi la méthode eulérienne, qui est plus simple à implémeenter et moins couteuse en complexité. La méthode lagrangienne peut rapidement exploser en temps de calcul en fonction du nombre de particules.
- Donner le cahier des charges détaillé

  Le cahier des charges du projet, réalisé en décembre 2025, fixait le cadre de développement, et définissait les objectifs, les contraintes et les fonctionnalités (minimales) à implémenter. Vous trouverez à la suite une synthése de celui-ci.
  - Le projet consistait à développer un rendu interactif permettant la visualisation et la manipulation en temps réel d'une simulation de fluide en 2D. Comme précisé lors de la réunion avec notre responsable de projet, la simulation devait être interactive et comporter une interface graphique. Nous avions fait le choix de développer ce projet en C++ avec un affichage en OpenGL.
  - L'objectif principal était d'obtenir une zone de rendu avec d'un coté la simulation et de l'autre une fenêtre de gestion des paramètres. Lors de la première réunion nous avions également déterminé le périmètre du projet. Nous avions décidé que la simulation reposerait sur une approche Eulerienne du fluide dans une grille 2D de taille 100×100.
  - Nous avions également pensé aux fonctionnalités que nous pourrions rajouter comme la connexion bluetooth à un téléphone permettant de gérer la simulation ou encore l'ajout de musique en fonction des forces du fluide. Enfin, nous avions imaginé une organisation moudlaire (calcul du fluide, interactions, affichage), permettant une répartition optimale des tâches.

2 Développements Logiciel : Conception, Modélisation, Implémentation

8 à 15 pages

1. Technologies utilisées
   — langage de programmation : c++ (justification pour l'optimsation, etc)
   — openGL (affichage : api graphique)
   — Imgui (facile d'utilisation)
2. Présenter les développements logiciel réalisés dans le cadre du projet. (fenetre, zone de rendu et panneau de parametres, et zone de rendu (shader, openGL))
   Dans cette section, nous allons présenter l'évolution de l'affichage de notre projet au cours du semestre, depuis les premiers affichages (grille de chaleur) jusqu'à la version finale de notre simulation de fluide. Nous nous intéresserons successivement aux différentes étapes de notre projet, grille de chaleur, affichage du fluide, fenêtre de paramètres et ajouts d'obstacles.

Évolution 1 : premier affichage d'une grille de chaleur (voir figure1)

Dans notre première version, l'objectif était d'avoir un premier affichage graphique avec OpenGL. Nous avons donc initialisé une fenêtre et une zone de rendu OpenGL affichant une grille. Cette grille prend des valeurs et les affiche sous forme de carte de chaleur. Notons la pertinence des couleurs, nous avons utilisé plusieurs teintes de rouges pour garder une cohérence avec la diffusion de chaleur (à coté d'une case très chaude se trouve une case un peu moins chaude mais pas froide, donc moins rouge mais pas bleue). Cette étape nous a permis de tester la création de la fenêtre, la boucle principale et l'utilisation de shaders pour colorer chaque cellule en fonction d'une valeur.

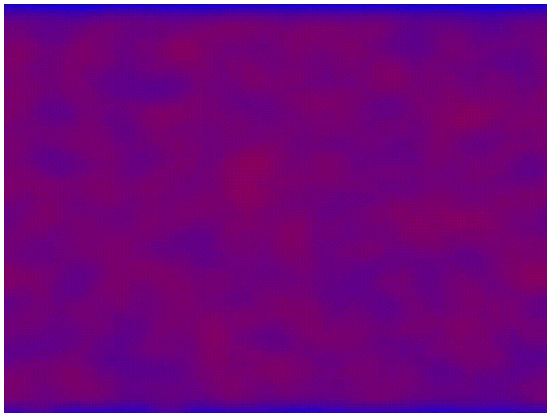


FIGURE 1 – Première version : affichage d'une grille de chaleur.

Évolution 2 : première simulation de fluide (voir figure2)

Dans un second temps, nous avons implémenté la méthode de Jos Stam *Real-Time Fluid Dynamics for Games* qui nous a permit d'avoir un beau rendu en 2D et une première simulation de fluide. Cette méthode repose sur une représentation du fluide dans une grille Eulerienne (voir section ....). La zone de rendu affiche donc la densité du fluide qui est mise à jour à chaque itération de la boucle de simulation. Dans cette version, les paramètres (couleur, densité, intensité) sont modifiables uniquement dans le code lui-même, c'est-à-dire qu'ils sont hard-codés. Cette version est intéressante car elle nous permet d'observer une simulation de fluide en temps réel.

Évolution 3 : ajout du panneau de paramètres (voir figure3)

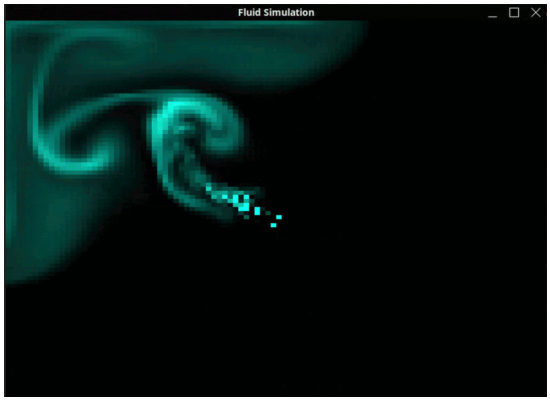


FIGURE 2 – Deuxieme version : affichage d'un fluide.

Nous avons ensuite ajouté un panneau de paramètres en utilisant la bibliothéque ImGui. Ce panneau nous permet de modifier en temps réel, lors de la simulation, différents paramètres tels que la densité, la couleur ou encore la direction du fluide. Les valeurs saisies par l'utilisateur sont envoyées au module de simulation qui met à jour la grille. Notons que ce panneau peut s'enlever et se remettre en pressant la touche "h".

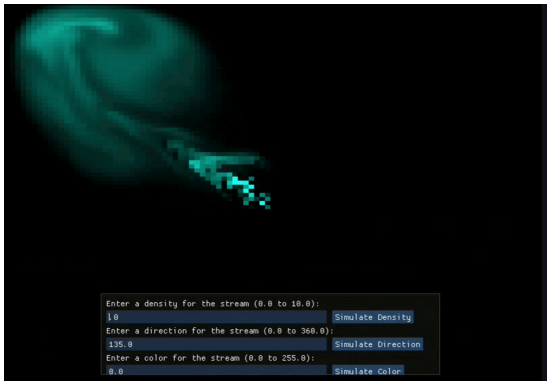


FIGURE 3 – Troisième version : ajout d'un panneau de paramètres.

Évolution 4 : gestion des obstacles

Enfin nous avons étoffé notre simulation en ajoutant des obstacles. Nous avons donc rajouté au panneau de paramètres les différents outils pour contrôler ceux-ci. En effet, l'utilisateur peut activer ou désactiver les obstacles et changer leur forme, nous pouvons également rajouter plusieurs obstacles. Lors de la simulation, nous pouvons les déplacer avec le curseur de notre souris. Nous avons également implémenté deux modes d'injection du fluide : un point ou une ligne qui modifient le comportement du fluide. Par la même occasion nous avons rajouté un mode d'affichage représentant la chaleur du fluide, l'utilisateur choisi sur le panneau de paramètres si le fluide sort chaud ou froid (rouge ou bleu).

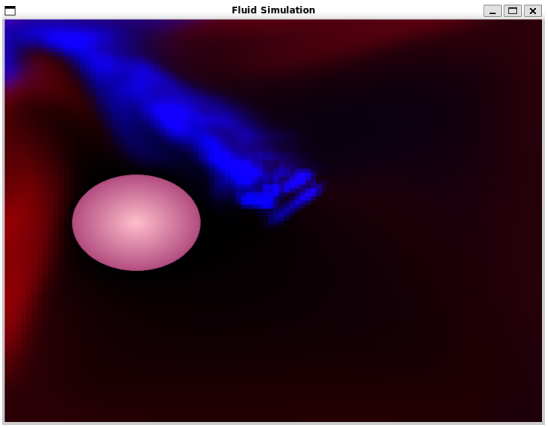


FIGURE 4 – Quatrième version : ajout d'un obstacle.

3. Présenter les principaux modules du logiciel développé dans le cadre du projet. Utiliser le langage UML pour la modélisation : donner le diagramme de cas d'utilisation et le diagramme des classes.

   Dans cette partie nous allons nous intérresser à l'architecture de notre projet.
   Le diagramme UML (voir figure5) suivant décrit l'organisation du projet. Nous pouvons clairement distinguer les principaux modules :
   — Données : module qui regroupe les structures représentant la grille, les cellules et les particules (nous vons également inclu la classe Simulation pour des raisons de clarté) ;
   — Fluid Solver : qui implémente les étapes de la simulation ;
   — Zone de rendu : ce module permet l'affichage en OpenGL ;
   — Fenêtre de paramètres : interface Imgui ;
   — Interaction utilisateur : ce module traite les actions du clavier ou de la souris ;
   — Gestion des obstacles : il a pour but de réunir ce qui touche aux obstacles, la création et le déplacement de ceux-ci.

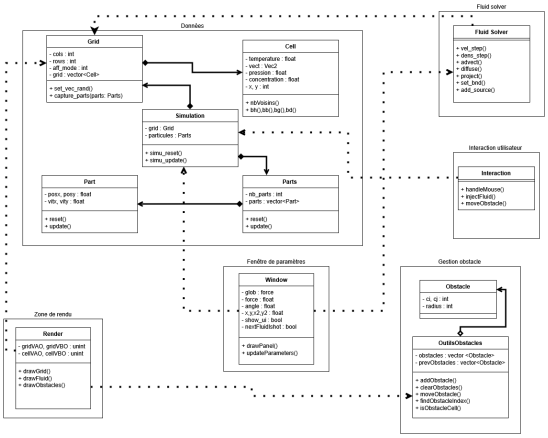


FIGURE 5 – Diagramme UML

   Ce diagramme permet de visualiser les dépendances entre modules et la structure de la simulation.
   — PARTIE A : terminé, bien implémenté
   — grille de départ (axel) (description des modules dans l'état actuel/final du projet)
   — obstacle (satine)

   Dans un premier temps, nous avons implémenté la création d'un **unique obstacle**, en modifiant notament la fonction de gestion des bords afin qu'elle reconnaise les obstacles. Nous pouvions également afficher ou non cet obstacle à l'aide d'un **bouton on/off** qui nous rammener a l'utlisation ou non de la fonction lancant la creation de l'obstacle. Par la suite, nous avons rendu cet obstacle le plus **interactif** possible en offrant à l'utilisateur la possibilité de

le déplacer à l'aide du curseur de la souris.

Nous avons ensuite enrichi notre simulation en ajoutant la possibilité de gérer **plusieurs obstacles simultanément**, ainsi que de choisir leur forme parmi un **cercle**, un **cœur** ou une **étoile**. Pour cela, nous avans implementer un curseur comptant le nombre d'obstacle par forme ainsi qu'un bouton +/- sur chaque forme possible afin de pouvoir ajouter ou supprimer les differents obstacles. Les obstacles sont stockés dans une unique **liste** contenant leurs coordonnées (x, y), rayon et forme, ce qui permet de les manipuler facilement (ajout, suppression, sélection et déplacement) tout en créant un ordre de priorité. L'utilisateur peut ainsi ajuster dynamiquement le nombre d'obstacles via une interface dédiée.

Afin de représenter précisément les différentes formes et leurs interactions avec le fluide, nous avons implémenté des fonctions mathématiques permettant, pour chaque point (x, y), de déterminer sa position par rapport à un obstacle (intérieur, extérieur ou bord). Par exemple, dans le cas d'un cercle de centre (cx, cy) et de rayon r, la distance au bord est donnée par :

$$d(x, y) = \sqrt{(x - cx)^2 + (y - cy)^2} - r$$

Ainsi, si $d < 0$, le point est à l'intérieur de la boule (donc le fluide ne peut pas y pénétrer), si $d = 0$, il est sur le bord (donc le fluide est en contact avec l'obstacle), et si $d > 0$, il est à l'extérieur (donc le fluide peut y pénétrer).

Pour des formes plus complexes comme le cœur, nous utilisons une représentation définie à l'aide d'un paramètre $t \in [0, 2\pi]$:

$$x(t) = 16 \sin^3(t)$$

$$y(t) = 13 \cos(t) - 5 \cos(2t) - 2 \cos(3t) - \cos(4t)$$

Ces coordonnées sont ensuite normalisées, redimensionnées et translatées afin de positionner correctement le cœur dans la simulation. Cette approche permet d'obtenir une forme realiste et cohérente.

De même, le triangle est conçu à partir de deux triangles rectangles.

Enfin, pour améliorer le plus possible le réalisme de la simulation, nous avons mis en place une gestion des interactions entre le fluide et les obstacles. En utilisant en particulier, la fonction `obstacleNormal` qui permet de calculer la **normale** au bord de chaque obstacle, c'est-à-dire la direction perpendiculaire à sa surface. Cette information est essentielle pour simuler correctement les phénomènes de réflexion et de rebond du fluide. Ainsi, le fluide réagit de manière cohérente à la forme des obstacles, rendant la simulation realiste et représentatif.

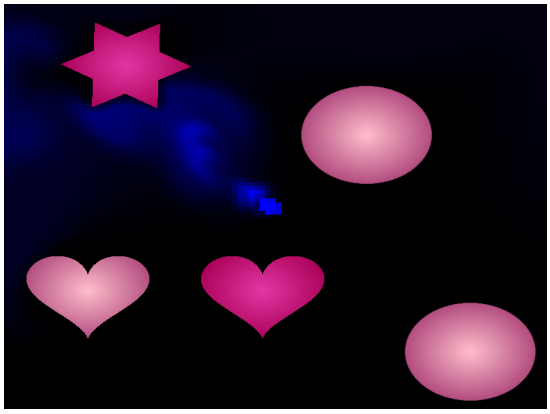


FIGURE 6 – Ajout de plusieurs obstacles.

— chaleur (saurane)
— Point ou ligne : Nous avons ajouté dans l'interface un paramètre permettant de choisir la forme de la source de fluide. L'utilisateur peut injecter le fluide soit en un point unique, soit le long d'une ligne définie par deux coordonnées dans la grille (soit 4 points).
En mode point, la densité et les forces sont appliquées uniquement à la cellule correspondant à la position sélectionnée.
En mode ligne : nous déterminons simplement le nombre d'étapes nécessaires pour relier les deux points $(i_0, j_0), (i_1, j_1)$ en utilisant que le nombre d'étapes nécessaire est :
$step = max(|i_1 - i_0|, |j_1 - j_0|)$. Lors ces étapes, nous parcourons, simplement toutes les cellules situées entre les deux positions sélectionnées, en suivant le segment qui les relie. Chaque cellule rencontrée reçoit la densité et la force, ce qui permet de créer une source linéaire.
— calcul d'une simulation, le fluide, physique
— PARTIE B : pas terminé
— bluetooth, lagrangien, musique, etc

4. Décrire les fonctionnalités de l'interface graphique implémentée (champs de simulation, bouton, affichage de l'obstacle).
— description des boutons et des interactions possibles

5. Décrire le format des données en entrée ou encore les conventions utilisées pour les entrées de vos programmes.(représentation du fluide, grille, stackage des paramètres)
— fluide stocké dans une grille
— grille composée de cellules
— cellules contiennent les paramètres de la simulation (densité, chaleur, vitesse)
— expliquer ce que représentent ces paramètres

6. Statistiques du projet
En termes de statistiques, notre projet est composé de certaines classes principales : Cellule, Grille, Obstacle, etc. Il est composé de 22 fichiers (cpp et h), et contient plus de 2000 lignes de code. Le projet est organisé en plusieurs modules, tels que le module de gestion des obstacles, le module de gestion de la chaleur, etc.

3 Algorithmes et Structures de Données

2 à 3 pages

1. Présenter les principaux algorithmes implémentés. En illustrer le fonctionnement avec des exemples.
Dans cette section, nous allons présenter et analyser l'un des algorithmes principaux du projet : `vel_step`. Pour cela, nous allons détailler 3 algorithmes essentiels : advection, diffusion et projection. `vel_step` est l'algorithme qui fait évoluer la vitesse du fluide à chaque itération de la simulation.

```
Algorithme : vel_step
    Début
        ajouter_source(N, u, u_prev, dt)
        ajouter_source(N, v, v_prev, dt)

        Echanger u et u_prev
        diffuse(N, 1, u, u_prev, visc, dt)
        Echanger v et v_prev
        diffuse(N, 2, v, v_prev, visc, dt)

        project(N, u, v, u_prev, v_prev)

        Echanger u et u_prev, Echanger v et v_prev
        advect(N, 1, u, u_prev, u_prev, v_prev, dt)
        advect(N, 2, v, v_prev, u_prev, v_prev, dt)
```

```
        project(N, u, v, u_prev, v_prev)
    Fin
```

L'Advection

La Diffusion

Comme l'advection, la diffusion est un processus physique qui décrit la propagation d'une substance. Nous avons donc choisit de l'implementer en pseudocode comme suit :

```
fonction DIFFUSE(N, x, x0, diff, dt)
    a ← dt × diff × N²

    pour k de 1 à 20 faire
        pour i de 1 à N faire
            pour j de 1 à N faire
                x[i,j] ← ( x0[i,j]
                        + a × ( x[i-1,j] + x[i+1,j]
                                + x[i,j-1] + x[i,j+1] ) )
                        / (1 + 4a)
            fin pour
        fin pour

        appliquer_conditions_aux_limites(x)
    fin pour

    retourner x
fin
```

L'algorithme de diffusion utilise l'equation aux deriées partielle qui suit :

$$\frac{\partial x}{\partial t} = D\nabla^2 x$$

où :

- x est la quantité diffusée (densité, température),
- D est le coefficient de diffusion,
- ∇^2 est le Laplacien.

En utilisant la discretisation utiliser par Jos Stam on obtient :

$$x^{t+1} = x^t + \Delta t\, D\, \nabla^2 x^{t+1}$$

Ce choix est essentiel pour garantir la stabilité numérique, même pour de grands pas de temps. Sur une grille 2D, le Laplacien discret est approximé par :

$$\nabla^2 x_{i,j} \approx x_{i-1,j} + x_{i+1,j} + x_{i,j-1} + x_{i,j+1} - 4x_{i,j}$$

En remplacant la formule de Jos Stam par l'approximation du Laplacien, on obtient :

$$x_{i,j}^{t+1} = \frac{x_{i,j}^0 + a\left(x_{i-1,j}^t + x_{i+1,j}^t + x_{i,j-1}^t + x_{i,j+1}^t\right)}{1+4a}$$

avec :

$$a = \Delta t \cdot D \cdot N^2$$

D'après Gauss-Seidel, l'équation précédente correspond à un système linéaire de la forme :

$$(I - a\Delta)x = x_0$$

Ce système est résolu par une méthode itérative de Gauss-Seidel. Elle consiste à :

— parcourir la grille plusieurs fois,
— mettre à jour chaque cellule immédiatement avec les nouvelles valeurs disponibles,
— répéter jusqu'à convergence.

Cette méthode accélère la convergence et permet une simulation stable et efficace.

Ainsi, on peut interpréter la diffusion comme un processus qui rend les valeurs de plus en plus uniformes : Une zone où la quantité est très concentrée se diffuse progressivement vers les cellules voisines. Au fil des itérations, cela rend la répartition de plus en plus uniforme, ce qui est indispensable pour obtenir des simulations de fluides réalistes.

La Projection

Après avoir appliqué les algorithmes d'advection et de diffusion, l'algorithme de projection implémente la décomposition de Hodge. C'est un algorithme qui permet de rendre le champ de vitesse du fluide incompressible et qui force la vitesse à conserver la masse. C'est une propriété importante des fluides réels.
Visuellement, la projection force l'écoulement à avoir de meilleures courbes, et évite les écoulements en ligne droite.
La décomposition de Hodge nous dit que tout champ de vecteur de vitesse est la somme d'un champ conservant la masse et d'un champ de gradient. L'idée est de rendre le champ de vitesse conservateur de masse lors de la dernière étape en soustrayant le champ de gradient du champ de vitesse actuel.

```
Algorithme : Projection
    Entrées : int N, float * u, float * v, float * div, float * p
    Sorties : u et v modifiés pour être incompressibles

    Début
        h = 1.0 / N

        # ÉTAPE 1
        Pour chaque cellule (i, j) de la grille :
            div[i, j] = -0.5 * h * (u[i+1, j] - u[i-1, j] + v[i, j+1] - v[i, j-1])
            p[i, j] = 0
        Fin Pour
        Appliquer les bords pour 'div' et 'p'

        # ÉTAPE 2
        Répéter 20 fois :
            Pour chaque cellule (i, j) de la grille :
                p[i, j] = (div[i, j] + p[i-1, j] + p[i+1, j] + p[i, j-1] + p[i, j+1]) / 4
            Fin Pour
            Appliquer les bords pour 'p'
        Fin Répéter

        # ÉTAPE 3 :
        Pour chaque cellule (i, j) de la grille :
            u[i, j] = u[i, j] - 0.5 * (p[i+1, j] - p[i-1, j]) / h
            v[i, j] = v[i, j] - 0.5 * (p[i, j+1] - p[i, j-1]) / h
        Fin Pour
        Appliquer les bords pour 'u' et 'v'
    Fin
```

L'algorithme de projection est composé de 3 étapes :

— Etape 1 : calcul de la divergence : pour chaque cellule de la grille, on évalue la divergence discrète

$$div_{i,j} = -\frac{h}{2}(u_{i+1,j} - u_{i-1,j} + v_{i,j+1} - v_{i,j-1}) \tag{1}$$

où $h = 1/N$ est la taille d'une cellule de la grille, et u et v sont les composantes de la vitesse du fluide.

— Etape 2 : résolution de l'équation de Poisson
On résout itérativement l'équation $\nabla^2 p = div$ par 20 itérations de relaxation de Gauss-Seidel, où p est un champ de pression :

$$p_{i,j}^{(k+1)} = \frac{1}{4}(div_{i,j} + p_{i+1,j}^{(k)} + p_{i-1,j}^{(k)} + p_{i,j+1}^{(k)} + p_{i,j-1}^{(k)} - div_{i,j}) \tag{2}$$

On réutilisa exactement le code de la diffusion pour assurer la cohérence du code et éviter les redondances.

— Etape 3 : correction du champ de vitesse
pour chaque cellule de la grille, on corrige la vitesse en soustrayant le gradient du champ de pression :

$$u'_{i,j} = u_{i,j} - \frac{1}{2h}(p_{i+1,j} - p_{i-1,j}) \tag{3}$$

$$v'_{i,j} = v_{i,j} - \frac{1}{2h}(p_{i,j+1} - p_{i,j-1}) \tag{4}$$

2. Évaluer la complexité théorique en temps des algorithmes présentés
Les algorithmes d'advection, de diffusion et de projection nous permettent de définir l'algorithme `vel_step`. Sa complexité en temps est déterminée par les 3 algorithmes principaux qui la composent :
— Advection :
— Diffusion :
L'algorithme parcourt une grille de taille $N \times N$ à chaque itération, ce qui représente un coût de N^2. Comme le nombre d'itérations est constant (20), la complexité totale est en $\mathcal{O}(N^2)$.
— Projection :
L'algorithme de projection est composé de 3 étapes :
— Etape 1 : calcul de la divergence : N^2 opérations
— Etape 2 : résolution de l'équation de Poisson : $20 \times N^2$ opérations
— Etape 3 : correction du champ de vitesse : N^2 opérations
Ainsi, la complexité totale de l'algorithme de projection est en $\mathcal{O}(N^2)$.
Ce qui nous donne une complexité totale de ... pour l'algorithme `vel_step`.

4 Analyse des résultats

2 à 4 pages L'analyse expérimentale sera faites dans cette section. Elle sera plus ou moins importante dans votre projet comme pour les deux points précédent, cette section devra donc être développée en conséquence.

1. Illustrer les performances ainsi que l'efficacité du logiciel implémenté à l'aide de graphiques.
— calcul de complexités en fonction de la taille de la grille, etc (plot sur python), nombre d'obstacles, étape d'avection/relaxation (changer les paramètres et comparer)
2. Analyser (et comparer, si plusieurs) les performances des solutions implémentées.
— eulerien vs lagrangien (uniquement eulerien a été implémenté)
— analyser les performances visuelles en fonction du nombre d'obstacles, densité, etc (mettre des screenshot) On peut voir l'évolution de notre projet a travers son evolution visuelle. mais egalement c'est peformance en fonction de différents paramètres tels que la densité du fluide ou le nombre d'obstacles.

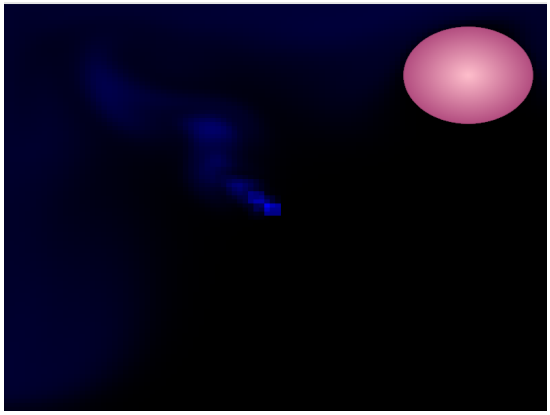


FIGURE 7 – Simulation avec un obstacle et toute les proprite du fluides non modifiées

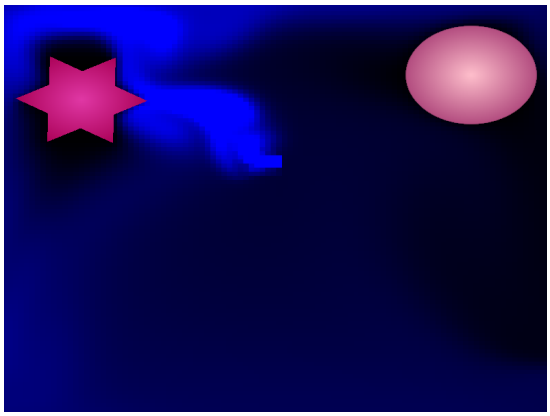


FIGURE 8 – Simulation avec 2 obstacles et une densité plus élevée(5)

On constate une meillieur visibilite des obstacle et du fluide ainsi que sa direction.

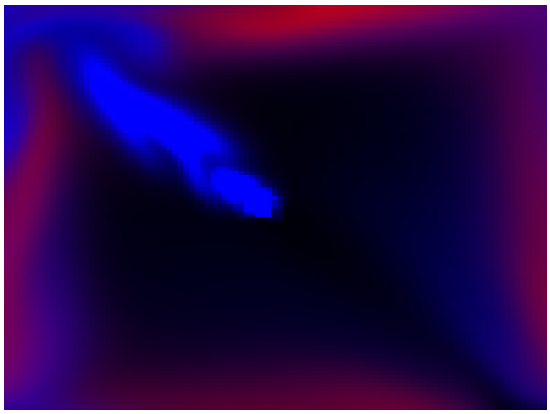


FIGURE 9 – Simulation avec alternace etre le rouge et le bleu pour representer la chaleur du fluide

On remarque que le fluide sortant ce melange assez facilement en creant des degradés de rouge et de bleu.

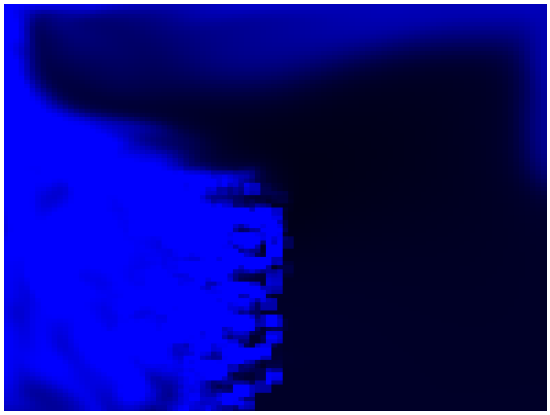


FIGURE 10 – Simulation avec sortie du fluide en ligne horizontale

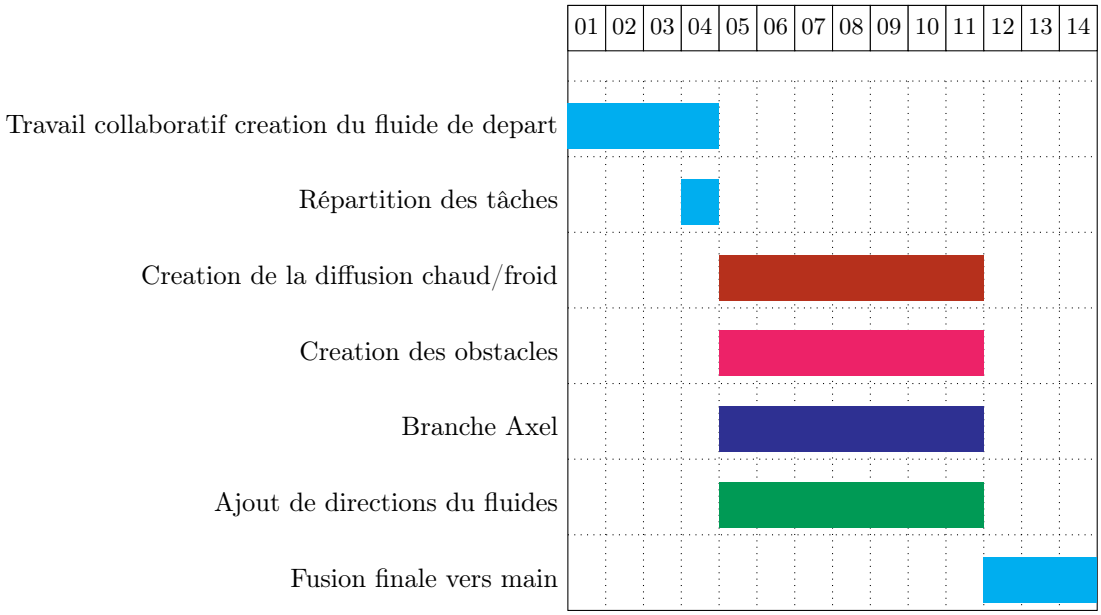
On constate un fort fluite remplissant vite le cadre comparer au point des image au dessus.

5 Gestion du Projet

(1-2 pages)

1. Présenter la gestion du projet et les documents de planification rédigés (par exemple, le diagramme de Gantt).
   — répartition des roles
   Durant le projet, nous avons dans un premier temps travaillé ensemble pour la mise en place de la grille de départ et la création du fluide ainsi que pour l'affichage de celui-ci. Une fois cette partie terminée, nous avons ensuite décidé de nous répartir les tâches pour être plus efficace. Ainsi, Gaelle a permis de déplacer le fluide dans diverses directions et a partir de divers points, Satine a travailler sur les obstacles, Axel à généré le code ImGui afin de pouvoir modifier les différents paramètres du fluides(densité, couleur,...) et le Bluetooth, et Saurane a travailler sur la diffusion de la chaleur.
   — diagramme de Gantt
   Nous avons réalisé un diagramme de Gantt pour planifier les différentes étapes du projet.

Organisation du travail - Diagramme de Gantt



Légende :
- Gaelle
- Satine
- Axel
- Saurane
- tout le monde (travail collaboratif sur la branche main)

— branches github, commits

Pour la mise en place d'un travail d'équipe optimale, nous avons utilisé GitHub. Lorsque nous travaillions ensemble, nous nous placions sur la branche main afin que chacun puisse accéder à ce que les autres ont fait facilement, puis lorsque nous avons choisi de nous repartir les taches nous nous sommes tous mis à travailler sur des branches différentes (Gaelle, Satine, Axel,Saurane) afin de pouvoir travailler et enregistrer notre travaille sans risquer de causer des conflits ou d'enregistrer quelque chose qui ne fonctionne pas. Une fois notre programme complet nous le mettions alors sur le main afin que tout le monde puisse y accéder. Nous avons également veillé à faire des commit réguliers et descriptifs pour suivre l'évolution du projet et faciliter la collaboration.

2. Discuter les changements majeurs effectués en cours de projet.
   — passage de la grille de départ à la nouvelle grille (pourquoi ?)
   — la façon d'implémenter les obstacles a été modifiée quand on est passé de l'affichage d'uun obstacle à plusieurs, est-ce que ça a impacté les autres modules ? pourquoi ? comment ?

   Pour la creation d'obstacle, il a fallut dans un premier temps réadapter la fonction qui délimite les bords du fluide afin que celui-ci prenne en compte les divers obstacles, ce qui a créé de nombreux problèmes (la disparition du fluide à travers l'obstacle ou la fuite du fluide à travers les bords), ou encore des problèmes de dépendance entre les différents obstacles lorsque nous avons voulu ajouter plusieurs obstacles mobiles. De plus, l'implémentation d'obstacles ayant une autre forme que la boule (le premier obstacle programmer) pose de nombreux problèmes notamment dans la partie de délimitation du fluide ou encore dans la partie d'affichage.

   Pour résoudre ces problèmes il a fallu revoir le code en changeant la façon de voir les obstacles mais également en les implémentant de manière différente. Ainsi, les obstacles ont été rangés dans une liste afin que chaque obstacle soit reconnaissable et manipulable facilement. Ils sont ainsi sélectionnés à partir de cette liste lorsqu'il faut les afficher, et c'est le dernier

obstacle de la liste qui est supprimé lorsque l'on désire en retirer un. Cette organisation a également permis de mieux gérer les interactions entre plusieurs obstacles et de réduire les conflits lors de leurs déplacements.

Enfin, cette nouvelle structure a facilité l'ajout de formes différentes et leur gestion dans la simulation, en rendant le système plus flexible et plus stable dans le temps.

— implémentation de la fenetre pour rentrer les paramètresuu

6 Bilan et Conclusions

Indiquer les fonctionnalités mises en œuvre par rapport au cahier des charges de départ, les points ouvertes. Les perspectives du projet décrivent ce que l'on pourrait ajouter, modifier, réécrire afin d'améliorer le projet.

1. Fonctionnalités rélisées qui étaient présentent dans le cahier des charges
   — interface graphique avec une fenetre d'affichage du fluide et des curseurs
   — simulation du fluide en 2D
   — possibilité d'agir sur le fluide avec des curseurs
   — possibilité d'ajouter des obstacles
2. Fonctionnalités non rélisées qui étaient présentent dans le cahier des charges
   — possibilité de controler le fluide à distance via un smartphone (bluetooth)
   — intégrer un mode musique qui fait réagir de fluide en focntion du son
3. Perspecvtives du projet pour la suite : ce qu'on aurait pu implémenter avec plus de temps.
   — lagrangien
   — autres fonctionnalités possibles avec les obstacles, etc

Références

[1] Donald E. Knuth (1986) *The TEX Book*, Addison-Wesley Professional.

[2] Leslie Lamport (1994) *LATEX : a document preparation system*, Addison Wesley, Massachusetts, 2nd ed.